



Un futuro compatible con la vida

Informe Técnico de Plataforma

Prueba Técnica · Software Engineer

Loth Mejía Martínez

Diseño e implementación de una plataforma geoespacial para registrar, consultar y analizar intervenciones territoriales con API, mapas interactivos y consultas en lenguaje natural asistidas por IA.

Demo en vivo	https://test-toroto.lobami.lat
Repositorio	https://github.com/lobami/toroto-challenge
Usuario demo	toroto
Contraseña	toroto573
API base	https://api-test-toroto.lobami.lat

01.

Qué construí y por qué

Este documento explica las decisiones técnicas que tomé al construir la plataforma. No es un manual de usuario — es el razonamiento detrás de cada elección: por qué PostGIS sobre otras alternativas, cómo diseñé la capa de IA para que nunca genere SQL, y dónde están los límites actuales del sistema.

El reto pedía una API geoespacial con almacenamiento, consultas espaciales y, opcionalmente, una capa de IA. Decidí implementar todo — incluyendo todos los bonus — porque cada pieza refuerza la siguiente: el modelo de datos bien indexado hace posibles las consultas rápidas; las consultas deterministas hacen posible la capa de IA sin riesgo de inyección o alucinaciones.

Acceso rápido

URL frontend	https://test-toroto.lobami.lat
Login	usuario: toroto / contraseña: toroto573
API base	https://api-test-toroto.lobami.lat
Repositorio	https://github.com/lobami/toroto-challenge

02.

Cumplimiento de requerimientos

Todos los requerimientos base fueron implementados. Los ítems bonus también fueron completados en su totalidad.

OK	Almacenar registros con coordenadas	POINT(lon lat) en PostGIS + lat/lon float para serializar JSON
OK	Consultar estado de los datos	Filtros por status, tipo, región y rango de fechas
OK	Historial de intervenciones	Campo date + timestamp, filtros por mes y año
OK	Diseño orientado a escalar	Índices GIST en las 3 tablas con geometría, caché en memoria
OK	Nearest neighbor	ST_Distance con cast ::geography — metros reales sobre WGS84
OK	Within radius	ST_DWithin::geography — usa el índice GIST, sin full scan
OK	Within area / polígono	ST_Within contra polígonos almacenados en tabla areas
OK	Corrección geoespacial	Geography casting en todas las operaciones de distancia
OK	Separación almacenamiento / lógica	models/ → services/ → routers/ — sin SQL en routers
OK	Capa de IA (solo diseño pedido)	Implementada y funcionando en producción
OK	PostGIS (bonus)	PostgreSQL 16 + PostGIS, GeoAlchemy2 ORM
OK	Geometrías complejas GeoJSON (bonus)	Polígonos en GeoJSON convertidos a WKT para PostGIS
OK	Índexación espacial (bonus)	CREATE INDEX USING GIST en interventions, areas, reference_places
OK	Visualización en mapa (bonus)	MapLibre GL — puntos, polígonos, highlight de resultados, flyTo
OK	Ejemplos de uso sobre el dataset (bonus)	7 sample queries interactivos en la UI

03.

Arquitectura general

El sistema tiene tres zonas: edge (Cloudflare Pages), compute (VPS Docker) y servicios externos. La separación edge/compute permite escalar el frontend de forma independiente y mantener el backend detrás de Nginx sin exponer puertos.

Zona	Componente	Tecnología	Dirección
Edge	Frontend SPA	React 19 + MapLibre GL + Vite	test-toroto.lobami.lat
Edge	TLS + CDN	Cloudflare Pages	—
Compute	Reverse proxy	Nginx	api-test-toroto.lobami.lat
Compute	API REST	FastAPI + uvicorn	:8000 interno
Compute	Base de datos	PostgreSQL 16 + PostGIS	:5432 Docker
Externo	Modelo de IA	GPT-OSS-20B vía Fyra	api.fyra.im/v1
Externo	CI/CD	GitHub Actions	—

Capas internas de la API

Separación estricta: ningún router escribe SQL. La lógica espacial vive en services/, los modelos en models/, y los routers solo validan entrada (Pydantic) y delegan a services.

```
HTTP Request
|
+--> Middleware          <-- JWT auth + license check
|
+--> Router (routers/) <-- validacion Pydantic
|   |
|   +--/ask --> nl_query.py --> GPT-OSS-20B (LLM)
|   |                                     '--> heuristico regex (fallback < 1ms)
|   |
+--> Service (services/) <-- logica geoespacial
|
+--> Model (models/)      <-- SQLAlchemy + GeoAlchemy2
|
'--> PostgreSQL + PostGIS
```

04.

Modelo de datos

Cuatro tablas. Toda geometría tiene índice GIST. Los campos lat/lon float coexisten con la columna geom: lat/lon para serializar JSON rápido, geom como fuente de verdad espacial.

interventions		
id	STRING PK	Identificador único
type / subtype	STRING idx	Tipo y subtipo — gavión, zanja, reforestación...
status	STRING idx	planned in_progress completed validated
project_id	STRING idx	Proyecto al que pertenece
region	STRING idx	Región operativa
quality_score	FLOAT	Puntaje 0-1
date	DATE	Fecha de registro
lat / lon	FLOAT	Coordenadas para serializar JSON rápido
geom	GEOMETRY(POINT, 4326)	Fuente de verdad espacial — índice GIST
metadata/metrics	JSON	Atributos adicionales por tipo

reference_places		
key	STRING PK	Slug del lugar — jalisco, monterrey, cankun
name	STRING	Nombre legible
lat/lon	FLOAT	Centroide
type	STRING	state city
aliases	JSON []	Variantes para el parser NL
geom	GEOMETRY(POINT, 4326)	Para consultas espaciales futuras — GIST

areas		
area_id	STRING PK	Identificador del polígono
name	STRING	Nombre legible
geometry	JSON	GeoJSON original para el frontend
geom	GEOMETRY(POLYGON,4326)	Para ST_Within — índice GIST

users		
id	STRING PK	UUID v4
username	STRING UNIQUE	Nombre de usuario
hashed_password	STRING	Hash bcrypt 12 rondas
role	STRING	admin viewer
is_active	BOOLEAN	Control de acceso

05.

Consultas geoespaciales PostGIS

Nearest neighbor

El cast `::geography` obliga a PostGIS a calcular distancias sobre el elipsoide WGS84. Sin él, usa geometría plana y la distancia en México puede diferir varios km para puntos alejados.

```
SELECT i.*,
       ST_Distance(
         i.geom::geography,
         ST_SetSRID(ST_MakePoint(-100.316, 25.686), 4326)::geography
       ) AS distance_m
FROM interventions i
ORDER BY distance_m ASC
LIMIT 1;
```

Within radius

`ST_DWithin::geography` filtra candidatos con bounding box esférico antes de calcular la distancia exacta, aprovechando el índice GIST sin full scan.

```
SELECT i.*,
       ST_Distance(i.geom::geography, ref::geography) AS distance_m
FROM interventions i,
     ST_SetSRID(ST_MakePoint(-103.320, 20.700), 4326) AS ref
WHERE ST_DWithin(i.geom::geography, ref::geography, 5000)
      AND EXTRACT(month FROM i.date) = 3
      AND EXTRACT(year  FROM i.date) = 2026
ORDER BY distance_m ASC;
```

Within area (polígono)

Se usa `geometry` (no `geography`) porque los polígonos son pequeños y la diferencia plana/esférica es despreciable.

```
SELECT COUNT(*), array_agg(i.id) AS ids
FROM interventions i
JOIN areas a ON a.area_id = 'zone_c'
WHERE ST_Within(i.geom, a.geom);
```

Por qué Geography y no Geometry en distancias

Operación	Tipo usado	Razón
ST_Distance, ST_DWithin	::geography	Metros reales sobre WGS84 — esencial para México
ST_Within (polígonos)	geometry	Polígonos pequeños, diferencia plana/esférica despreciable
Inserción	WKTElement	GeoAlchemy2 convierte lat/lon a POINT automáticamente

06.

Capa de inteligencia artificial

El reto pedía diseño de la capa de IA. La implementé completa en producción. El principio central: el LLM extrae intención estructurada, nunca genera SQL. Esto tiene tres ventajas: (1) la consulta PostGIS es siempre determinista y auditable; (2) no hay riesgo de inyección SQL ni de joins inventados por el modelo; (3) si el LLM falla, el heurístico regex responde en menos de 1ms.

Flujo completo

```
Usuario: "intervenciones cerca de guanajuato"
|
+--> POST /ask {question} + Bearer JWT
|
+--> GPT-OSS-20B (si FYRA_API_KEY disponible)
|     system_prompt con esquema + ejemplos del dataset
|     responde: GeoIntent JSON
|
+--> heuristico regex (fallback)
|     "cerca" -> nearest_to_place
|     "guanajuato" -> lookup en cache de reference_places (0(1))
|     responde: GeoIntent JSON
|
+--> geospatial.py -> ST_Distance ORDER BY ASC LIMIT 1
|
'--> { intent, result: { intervention, distance_km } }
      Frontend: resultado + flyTo al punto en el mapa
```

Estructura de GeoIntent

```
{
  "intent":      "nearest_to_place",
  "place":      "guanajuato",
  "lat":        null, "lon": null, "radius_km": null,
  "area_id":    null, "status": null, "subtype": null,
  "start_date": null, "end_date": null, "region": null,
  "explanation": "Intervencion mas cercana al estado de Guanajuato."
}
```

Comparación de enfoques

Enfoque	Latencia	Riesgo principal	Adoptado
SQL generado por LLM	2-5s	Inyección SQL, joins incorrectos	No
Intent + funciones fijas	1-3s	Cobertura limitada de intents	Sí
RAG sobre documentos	3-8s	Complejidad operativa alta	No
Heurístico puro (fallback)	< 1ms	No maneja preguntas complejas	Sí (fallback)

07.

Autenticación y seguridad

JWT Bearer sobre todos los endpoints `/geospatial/*`. Implementado en middleware FastAPI — ningún router maneja auth directamente. Esto garantiza que cualquier endpoint nuevo quede protegido por defecto.

Algoritmo JWT	HS256, expiración en 24 h
Hash passwords	bcrypt directo (factor 12 rondas), sin passlib
Usuario demo	toroto / toroto573
Login endpoint	POST <code>/auth/login</code> (OAuth2 form-urlencoded)
Protección	Todos los endpoints <code>/geospatial/*</code> requieren Bearer token
Revocación	Rotar <code>JWT_SECRET</code> en GitHub Secrets invalida todos los tokens
Frontend	Token en <code>localStorage</code> ; 401 limpia sesión y recarga la página

Flujo de login

```
POST /auth/login username=toroto&password=toroto573
|
+--> SELECT user WHERE username = 'toroto'
+--> bcrypt.checkpw(plain, hashed) --> OK
'--> jwt.encode({sub, role, exp}) --> access_token

Requests posteriores:
Authorization: Bearer eyJhbG...
|
'--> Middleware: jwt.decode()
      --> valido --> continua al router
      --> invalido --> 401 Unauthorized
```

Nota sobre passlib

Durante el desarrollo detecté que `passlib[bcrypt]==1.7.4` es incompatible con `bcrypt >= 4.1` (lanza `ValueError` al inicializar). Decidí usar `bcrypt` directamente: menos dependencias, API idéntica, sin conflictos de versión.

08.

Infraestructura y despliegue

Componente	Tecnología	Estrategia
Frontend	Cloudflare Pages	vite build → deploy automático en push a main
Backend API	Docker + Nginx en VPS	tar bundle → SSH → docker compose up --build
Migraciones	Alembic	alembic upgrade head corre antes de uvicorn en CMD
CI/CD	GitHub Actions	2 workflows separados por ruta de archivos
Secrets	GitHub Secrets	JWT_SECRET, FYRA_API_KEY, DB creds — nunca en repo
Datos	Seed separado	Solo corre en primer deploy o reset intencional

Pipeline CI/CD

```
git push main
|
+--> deploy-frontend.yml (si cambio en apps/web/**)
|     npm install -> vite build -> Cloudflare Pages API
|
'--> deploy-backend-vps.yml (si cambio en apps/api/** o deploy/**)
    tar bundle (sin .git, sin node_modules)
    scp -> VPS
    ssh:
        extraer -> copiar .env.production -> nginx reload
        docker compose up -d --build
        CMD: alembic upgrade head && uvicorn :8000
```

Separación crítica: seed vs deploy

- **Deploy normal** — solo corre migraciones + rebuild. Los datos existentes no se tocan.
- **Seed** — docker cp del JSON + load_dataset.py borra e inserta todo. Solo se ejecuta la primera vez o ante un reset intencional.
- Esta separación evita que un deploy accidental borre datos de producción.

09.

Decisiones clave y trade-offs

Por qué PostGIS sobre otras alternativas

	PostGIS (elegido)	MongoDB Atlas	BigQuery Geo
Costo	Gratis (self-hosted)	\$\$\$	Pay-per-query
Consultas ST_*	Nativo y maduro	Limitado	Bueno pero lento
SQL completo	Sí	No	Sí
Control operativo	VPS propio	Managed	Serverless
Complejidad setup	Media	Baja	Baja

Caché de reference_places en memoria

Los 51 lugares de referencia (28 estados + 23 ciudades) se cargan en un diccionario Python al startup. El heurístico los consulta en O(1) sin round-trip a la base de datos. Trade-off aceptado: lugares nuevos insertados en DB no se reflejan hasta reiniciar el proceso. Razonable dado el volumen y la frecuencia de cambio.

Un solo commit en el repositorio

El historial de commits intermedios (debugging, fixes de compatibilidad, iteraciones de UI) no añade valor al evaluador. El repositorio final tiene un único commit limpio con el estado completo de la plataforma, acompañado de este informe para explicar el proceso y las decisiones.

10.

Limitaciones actuales y evolución futura

Limitaciones conocidas

Limitación	Impacto	Solución natural
Dataset sintético	No refleja distribución real de campo	Ingesta real desde app móvil o API de campo
Un solo nodo de DB	Sin HA — un corte baja el sistema	Read replica + PgBouncer
JWT sin refresh token	Sesión expira a las 24 h	Refresh token con rotación automática
Sin paginación en /interventions	Carga todos los registros al inicio	Cursor pagination + lazy loading en mapa
Heurístico solo en español	Preguntas en inglés fallan al LLM	Ampliar regex o NLP ligero

Camino de escalabilidad

HOY	SIGUIENTE	ESCALA MEDIA
-----	-----	-----
VPS unico	-> VPS + Read Replica	-> Kubernetes (HPA)
Docker Compose	-> PgBouncer pool	-> PostGIS particionado
	-> Redis cache	-> MVT tile server
ESCALA ALTA		

TimescaleDB (series temporales) + Citus (PostGIS distribuido)		
Vector store para RAG sobre historial de intervenciones		
Agente con tools geoespaciales (LLM function calling)		

Evolución de la capa de IA

- **Corto plazo:** ampliar de 6 a 15+ intents (por operador, calidad, tendencias temporales).
- **Mediano plazo:** RAG sobre historial — el LLM responde análisis narrativos sin SQL hardcodeado.
- **Largo plazo:** agente con herramientas geoespaciales — encadena múltiples queries y devuelve un análisis completo.
- **Independencia de proveedor:** la URL base y el modelo se configuran por variables de entorno. Cualquier API compatible con OpenAI funciona sin cambiar código.

Stack completo

Capa	Tecnología	Versión
Framework API	FastAPI + uvicorn	0.116 / 0.35
ORM	SQLAlchemy + GeoAlchemy2	2.0 / 0.17
Base de datos	PostgreSQL + PostGIS	16 / 3.x
Driver	psycopg	3.2
Migraciones	Alembic	1.16
Validación	Pydantic v2	2.11
Auth	PyJWT + bcrypt	2.10 / 4.2
HTTP client	httpx	0.28
Frontend	React + Vite	19 / 7
Mapas	MapLibre GL	latest

Deploy frontend	Cloudflare Pages	—
Deploy backend	Docker + Nginx	—
CI/CD	GitHub Actions	—
Modelo IA	GPT-OSS-20B vía Fyra	—